Build Phase 1 — Proctor Guide

Your Role

During Phase 1, your main job is to **unblock people quickly** and **protect the Playwright time later**. Most participants will hit the same few issues. This guide covers what to watch for and how to help.

Timeline Check-ins

Time Into Phase	Where They Should Be	Red Flag
----------	----------	----------
15 min	Have a plan, starting project setup	Still deciding on a
30 min	Project initialized, starting frontend	Stuck on tooling i
1 hour	Frontend form exists, starting API	No code written y
1.25 hours	API responds, wiring frontend to API	Still building the fo
1.5 hours	Minimal end-to-end flow works with basic validation and a small test-data fixture set exists	Frontend and API

Common Blockers & Quick Fixes

Environment / Setup Issues

- **Node.js not installed or wrong version:** Help them install via `nvm` or direct download. Don't let them spend more than 5 min on this.
- **Python virtual environment confusion:** `python -m venv venv` then activate. If it's taking too long, skip the venv and install globally.
- **Java/Maven/Gradle issues:** Spring Initializr (start.spring.io) is the fastest path. Download a pre-configured zip.
- **.NET SDK not found:** Verify with `dotnet --version`. If missing, this is a pre-req failure — help them install or pivot to another stack.

CORS Errors

The #1 blocker when frontend and API run on different ports. Quick fixes:

- **Express:** `npm install cors` then `app.use(cors())`
- **Flask:** `pip install flask-cors` then `CORS(app)`
- **Spring Boot:** `@CrossOrigin` annotation on the controller
- **.NET:** `builder.Services.AddCors()` in Program.cs

Have these snippets ready to paste.

Copilot Not Helping

- **Copilot giving irrelevant suggestions:** They likely don't have enough context. Suggest using `#file` references in chat, or opening relevant files so Copilot can see them.
- **Copilot generating too much at once:** Encourage smaller, focused prompts. "Add a POST endpoint" is better than "Build my entire API."
- **Plan mode not available:** Make sure they're on the latest Copilot extension version.

Frontend/API Not Connecting

- Check the port numbers match
- Check the fetch URL (common mistake: `localhost` vs `127.0.0.1`)
- Check that the API is actually running
- Check browser console for errors — paste them into Copilot Chat

When to Intervene

Do intervene if:
- Someone has been stuck for 10+ minutes on the same issue
- Someone is going down a rabbit hole (e.g., setting up a database, adding authentication, elaborate CSS)
- Someone is trying to overbuild the app instead of getting to a minimal testable flow
- Someone hasn't written any code after 30 minutes
- Someone is not using Copilot at all — gently remind them that's the point

Don't intervene if:
- They're making steady progress, even if slow
- They're experimenting with Copilot prompts and learning from the results
- Their approach is different from what you'd do — that's fine

Nudges for People Who Are Stuck

If someone is frozen on where to start:

> *"Start with the Plan agent — describe what you want to build and let Copilot outline the steps for you."*

If someone is over-thinking the architecture:

> *"Just get a form that submits to an endpoint and returns something. You can refactor later."*

If someone is spending too much time on styling:

> *"The form can be ugly — focus on getting data from the form to the API and back. The real learning goal later is Playwright."*

If someone is not using Copilot:

> *"Try describing what you need in Copilot Chat — something like 'Create a POST endpoint that validates this JSON.' See what it gives you."*

If someone is nearly done with core flow:

> *"Before Phase 2, create 3 fixtures now: one valid input, one missing-field input, and one invalid-value input."*

Phase 1 Success Criteria

At the end of Phase 1, each participant should have:

1. A frontend with a form (even if basic)
2. An API endpoint that receives data
3. The frontend and API are connected
4. Some validation logic runs and results display
5. A small fixture set ready for Playwright (valid + invalid examples)

It's OK if: The UI is ugly, there's only one validation rule, error handling is minimal.

It's not OK if: There's no working connection between frontend and API — participants need a minimal app ready for Playwright in Phase 2.

Facilitator One-Page: Variation Matrix (All Phases)

Use this during check-ins to recommend 2-3 experiments per phase without slowing teams down.

Phase	Pick 2-3 Variations	Fast Proctor Cu
---	---	---
Phase 1: working flow first	Test-First Variation, Selector Quality Challenge, Prompt Detail A/B Test	"Get one stable pa
Phase 2: correctness and confidence	Failure Injection Drill, Explain-Back Check, Role-Based Prompting	"Now stress the ap
Phase 3: polish and demo strength	Model Comparison, Refactor Pass, Demo Readiness Variant	"Choose the clean

30-Second Recommendation Script

If a participant asks what to do next, use this:

1. "Pick one reliability variation and one communication variation."
2. "Reliability means selector quality or failure injection."
3. "Communication means explain-back or demo-readiness script."
4. "If you still have time, do model comparison on one prompt."

Decision Rules by Situation

- If they are unstable or flaky: prioritize **Selector Quality Challenge** and **Failure Injection Drill**.
- If they are slow due to rework: prioritize **Prompt Detail A/B Test**.
- If they are close to demo time: prioritize **Refactor Pass** and **Demo Readiness Variant**.
- If they finish early: run **Model Comparison** on one completed checklist prompt and keep the better result.